HAW HAMBURG

Praktikum Grundlagen der systemnahen Programmierung
(BITS2-GSP)

Tipps für Aufgabe 2

Zeitmessung

Situation Die Winkelgeschwindigkeit ω des Drehgebers muss bestimmt werden

$$\omega = \frac{\triangle \varphi}{\triangle t}$$

Konsequenz Zeitspannen müssen gemessen werden

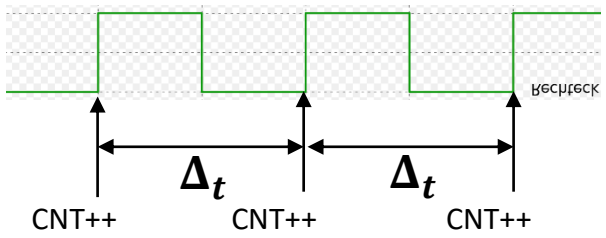
Basis (Hardware)-Timer (Hier: TIM2 von STM32F4xx)

- Timer haben unterschiedliche implementationsabhängige Funktionen (Genauigkeit, Vorwärts- / Rückwärtslauf, PWM, Input Capture, Kopplung von Timern, PreScaler Varianten, Auslösen von Events, …)
- Hier: ganz kleine Teilmenge dieser Aspekte zur Verdeutlichung der Idee eines Hardware-Timers

Genereller Aufbau Hardware Timer

CNT: 32 Bit Register

Oszillator



Unser Timer Modul: $\Delta_t = {}^{1}/_{90}\,\mu s$

Programm

```
uint32_t t1 = CNT; // Befehl b1
…
uint32_t t2 = CNT; // Befehl b2
```

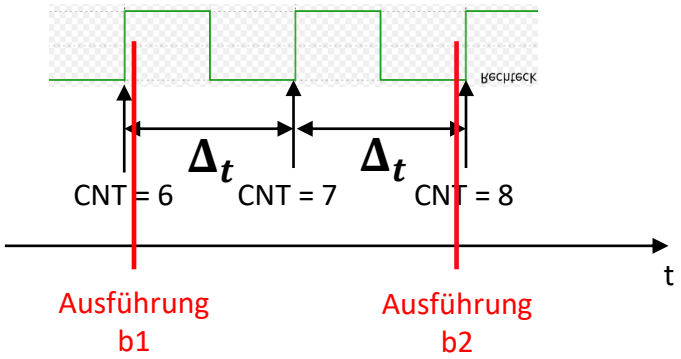
Ziel Bestimmung der Zeitspanne zwischen der Ausführung von b1 und b2

$$\mathrm{d} = (t2 - t1) * \Delta_t$$

Auflösung/Genauigkeit des Hardware-Timers

Programm:

```
uint32_t t1 = CNT; // Befehl b1
…
uint32_t t2 = CNT; // Befehl b2
```



Zeitspanne zwischen der Ausführung von b1 und b2 d = $(t2 - t1) * \Delta_t$

$= (7 - 6) * \Delta_t = \Delta_t$

Genauigkeit Δ_t

Verkleinerung von Δ_t - Erhöhung der Oszillatorfrequenz - erhöht die Genauigkeit

Oszillator hat eine Ungenauigkeit (Schwankung der Frequenz) – wie jede Uhr

- Ursachen: Technologie des Oszillators, Alter des Oszillators, Temperatur, …
- Konsequenz: Uhren (zweier Boards / Rechnern …) können auseinander laufen

Ein C Modul zur Zeitmessung

HAW HAMBURG

Warum ein Modul zur Zeitmessung?

- Kapselung eines „abgeschlossenen“ Funktionsblocks in einem Modul
  - Abstraktion – damit die Aufgabe überschaubar bleibt
  - Separat testbar (Modultest)
  - Arbeitsteilung im Team
- Verwendung der Funktionen an mehreren Stellen von A2
- Wiederverwendung des Moduls in A4/A5

Schnittstellen des Moduls

- Initialisierungsfunktion
- Funktion, die den aktuellen Zeitstempel liefert
- Funktion, die die Zeitspanne (in Sekunden) zwischen zwei Zeitstempeln berechnet

  return Wert: Zeitspanne in Sekunden (Typ double)

Ansteuerung Drehgeber

Was die Hardware liefert

Aktuelle Phase

Zuordnung Signale - Phase		
Kanal A	Kanal B	Phase
0	0	a
1	0	b
1	1	c
0	1	d

Zuordnung Phasenwechsel - Ereignis	
Phasenwechsel	Ergebnis
Phase b →Phase a	Rückwärtslauf
Phase b →Phase b	Keine Änderung
Phase b →Phase c	Vorwärtslauf
Phase b →Phase d	Fehler

- Zeitstempel, wann die aktuelle Phase ausgelesen wurde (indem die entsprechende Funktion des Moduls zur Zeitmessung aufgerufen wird)

Was “wir“ brauchen

- Zeitstempel des letzten Phasenwechsels
- Anzahl Phasenwechsel seit dem letzten Reset
  - Vorwärtsdrehung erhöht die Anzahl der Phasenwechsel
  - Rückwärtsdrehung reduziert die Anzahl der Phasenwechsel

  → negative Anzahl von Phasenwechseln möglich
- Aktuelle Drehrichtung (Vorwärtsdrehung, Rückwärtsdrehung)
- Fehlererkennung - ein Phasenwechsel wurde nicht erkannt

  Fehlerfall, wird durch Reset zurückgesetzt

Umsetzung

- Fehlererkennung und Drehrichtung kann man in einer FSM (finite state machine) verwalten
  - FSM ist ein Standardverfahren (s. AF und und in SE1 Vorlesung)
  - Wer FSMs kennt, kann sie gerne einsetzen
  - Hier einfacher alternativer Weg
- Anzahl der Drehimpulse wird in einer eigenen Variablen gepflegt
- Zeitstempel des letzten Phasenwechsels wir in einer eigenen Variablen gepflegt

- Diese Zustandsverwaltung wird in einem eigenen Modul implementiert (fsm.c)

Modulkonzept

- Ein Modulkonzept, das “Separation of Concerns“ beachtet und klare Schnittstellen definiert, ist entscheidend, damit Sie nicht aufgrund zu komplexer Funktionen und der Vermaschung der Komponenten unnötig Zeit verlieren.
- Zwei Module kennen wir schon:
  - fsm.c
  - Modul zur Zeitmessung
- Offensichtlich sind separate Module zur Display-Ausgabe, GPIO-Ausgabe und zum Einlesen von GPIOs sinnvoll.
- fsm.c liefert den aktuellen PulseCount und den Zeitstempel des letzten Phasenwechsels. Auf dieser Basis berechnet ein weiteres Modul Drehwinkel und Winkelgeschwindigkeit.